COMPUTATIONAL INTELLIGENCE

Politecnico di Torino
01URROV 01URRSM

2024-25

1

Computational Intelligence

https://github.com/squillero/computational-intelligence

2

General Information

Practicalities, curiosities, etc.

3

Contacts

Giovanni Squillero
giovanni.squillero@polito.it

Giuseppe Esposito
giuseppe.esposito@polito.it

https://t.me/polito_01urr

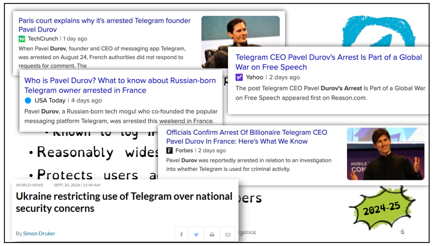
2024-25

4

Why Telegram?

- Not really private
  - E2EE not available in group chats
  - Closed source
  - Centralized infrastructure
  - Known to log IPs and other sensitive info
- Reasonably widespread among students
- Protects users against other group members
- Do not expose phone numbers

5



6

Syllabus

- Searching for...
  - a single state (in discrete and continuous spaces)
  - a sequence of actions (a.k.a., a path)
  - a formula (numeric, logic)
  - a policy (what to do in a given state)
  - a program (what to do, in general)
- Evolutionary Algorithms
- Multi-agent Systems
- Information representation

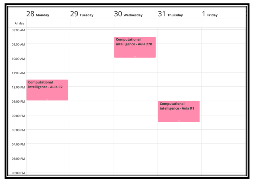
2024-25

7

Practicalities

8

Lectures



2024-25

9

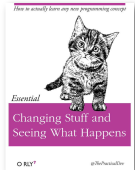
Labs

- 60h = 42 + 18 (roughly)
- No hard distinction between lectures and labs
- Bring your own laptop (or a friend with a laptop)
  - We will try to support online students
- Work in groups, help colleagues, and get help
- Ubuntu: A person is a person through other people

2024-25

10

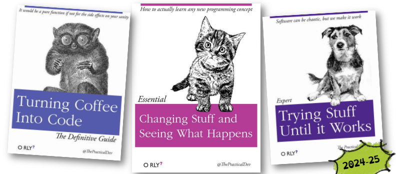
Official Textbook



2024-25

11

Official Textbook



2024-25

12

Cool Books

- S. Luke; "Essentials of Metaheuristics" (2nd edition) — freely available online
- A. E. Eiben, J. E. Smith; "Introduction to Evolutionary Computing" (2nd edition)
- M. Mitchell; "Artificial Intelligence: A Guide for Thinking Humans"
- S. Russell, P. Norvig; "Artificial Intelligence: A Modern Approach" (4th edition)

2024-25

Rules of Engagement

!

Rule 0: Don't cheat

- Solve together, submit alone
- Declare any help received and provide refs, e.g.,
  "I worked with Alice"
  "I copied the procedure from Bob's repo"
  "I found this idea on Stack Overflow"

Rules of Engagement

No chatting/texting during lectures

- You have 1 linguistic processor in your brain
- No mobiles
- Laptop in Zen mode
- You could exercise or play chess (if you are a professional), thou
- Please, don't waste your time

Rules of Engagement

Max lag is 24h

- To meet coursework deadlines, students that cannot attend in presence are required to participate online or to watch the recorded lecture within 24h
- After 1 day I can assume that everything <u>said</u> in class is known (without the need to write it)
- Caveat: Recorded lectures are **<u>not</u>** a valid substitute for in-person attendance

Rules of Engagement

Lectures are required but not sufficient

- Roughly 10h of lecture requires 15h of study (see "The academic credit system" in the *Student Guide*)
- Your duties:
  - Take notes (slides are not self contained)
  - Broaden the subjects
  - Run code and do experiment on your boxes (i.e., change stuff and see what happens)

Rules of Engagement

Face to face and Online behavior

- Limit the discussions to the course topics
- Show courtesy
- Respect different viewpoints and experiences
- Accept criticisms
- **Have a fair share of sense of humor**

Rules of Engagement

Questions

Exam

A GitHub repo is required

- GitHub will be used to certify submission times and for peer-review activities
- Register on GitHub, fill your info in the CryptPad shared sheet on **http://tiny.cc/ci24_github**
- Upload solution to Lab *N* in a repo named CI2024_labN
- **Note:** These instructions will not be repeated

2024-25

Why GitHub?

- De facto standard for version control and collaborative software development
- Large user base
- Strong brand recognition
  - Your future employers are likely to peek into your repos (be prepared!)
- Free (as in free beer, not free speech)

2024-25

Why CryptPad?

- Open source, free, private (self-claimed)
  - Note: You should not trust any web-based service to do what it claims to do
- No Google account, no email marketing
- Anonymous access through Tor
- Note: I'm using TinyCC's short links

2024-25

Exam

Timely participation: up to 6 points

- Post labs solution on GitHub before deadline
  - Note 1: **<u>all</u>** labs will be evaluated, including **Lab 0**
  - **Note 2:** failure to report your handle in the shared sheet or to name the directories correctly will result in no points being awarded in the exam
- Review peers' solutions on GitHub before deadline (**note:** We judge the **reviewer** based on their reviews, and we judge the **reviewed** based on how they implement peers' reviews)
- <u>Voluntarily</u> present sharp ideas in class

2024-25

Lab 0

- List your handle in http://tiny.cc/ci24_github
- Create a public repo named CI2024_lab0
- Upload a file named irony.md containing a joke, a pun, or a witty remark about the 1st lecture

DEADLINE: Wednesday, September 25 @ 14:00 UTC

Exam

Project Work: up to 12 points

- **The project work is mandatory**
- Students are encouraged to work in teams, but each student is individually responsible for the entire assignment
- The Project Work(s) will be assigned later

2024-25

Exam

Written Exam: up to 14 points

- **The written exam is mandatory**
- Computer-based written test using POLITO platform
- Open-ended questions covering all topics of the course and the project work
- Note: due to Politecnico's policies, students will need a laptop with a working LockDown browser (**test it and report any problem in advance!**)

2024-25

Exam

Oral Exam: an offset up to +/-99 points

- An oral exam may be required by the instructor (not by the student)
- Failure to attend will result in failure of the entire exam

2024-25

Exam

The PDF report

- **The report is mandatory**
- At least **168h** (i.e., 1 week) before the official exam date/time students must submit a PDF report detailing all their activities during the course (log like)
- The report is valid for 2024/25 only
- The report must be resubmitted if a student decide to retake the exam (sorry, but it's easier for us)
- Tip: write in Markdown (also with CryptPad), and generate PDF with Pandoc

2024-25

Exam

Caveat

- **Everything said in classroom is an integral part of the course and can be checked during the exam**
- **… including introductions, examples, notes, etc.**

2024-25

1. Partecipazione ai Laboratori:

- Completa tutti i laboratori e carica le soluzioni su GitHub entro le scadenze. I laboratori sono parte fondamentale della valutazione e vengono giudicati sia per la qualità del lavoro che per la revisione dei compiti dei tuoi colleghi.
- Ogni laboratorio contribuisce al punteggio complessivo, e partecipare tempestivamente può darti fino a **6 punti**.

2. Progetto:

- Il progetto finale è obbligatorio e può contribuire fino a **12 punti**. Assicurati di lavorare in gruppo se consentito, ma ogni studente è responsabile individualmente del proprio lavoro.
- Lavora su un progetto ben strutturato, mostrando l'applicazione di tecniche di *Computational Intelligence* e ottimizzazione.

3. Esame Scritto:

- L'esame scritto può darti fino a **14 punti**. Preparati su tutti gli argomenti del corso, inclusi gli algoritmi evolutivi, i sistemi multi-agente, la ricerca locale, e i problemi NP.
- L'esame sarà basato su domande aperte che copriranno sia il corso che il progetto, quindi studia a fondo il materiale.

Pop Quiz

- Picture yourself running in a street carrying a handful of hamsters
  - What problems do you expect?
  - How would you solve them?
  - Did you ever do it?
  - Did someone tell it to you?

Pop Quiz 2

- A.k.a., Wozniak's cup of coffee
- Would you be able to enter my house and prepare me a cup of coffee?

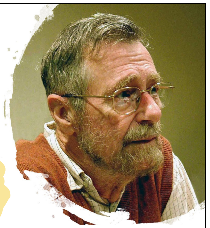
Pop Quiz 2

- A.k.a., Wozniak's cup of coffee
- Would you be able to enter my house and prepare me a cup of coffee?

Artificial Intelligence?

The question of whether a computer can think is no more interesting than the question of whether a submarine can swim

— Edsger Wybe Dijkstra (1930-2002)



Narratives

- Since 1996
  - AI has meant playing chess
- …
- Since 2013
  - AI has meant image recognition
- Since 2021
  - AI has meant image generation
- Since 2022
  - AI has meant text generation
- What will AI mean tomorrow?

Artificial Intelligence

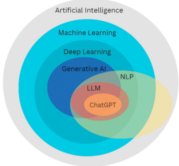
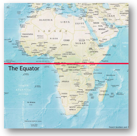


Image from: Ebtesam Ahmad Alomari, Unlocking the Potential: A Comprehensive Systematic Review of ChatGPT in Natural Language Processing Tasks

Pop Quiz

- How many countries are crossed by the equator line in Africa? Shout the answer!!!

- Understand the problem (text)
- Invent an algorithm
- Change domain (visual)
- Look up a map
- Execute the algorithm
- Change domain (speak)



9/23/24

Complexity and Intelligence

- Complex ≠ Intelligent
- The Game of Life, John Horton Conway (1970)
  - Repeated applications of simple rules may lead to unforseable results
  - Complex properties arise from simple laws

Generalization and Intelligence

- Intelligence measures an agent's ability to achieve goals in a wide range of environments
- AI is the science and engineering of making machines do tasks they have never seen and have not been prepared for beforehand

9/23/24

Generalization and Intelligence

- Local generalization
  - A.k.a., robustness
- Adaptation to known unknowns within a single task or well-defined set of tasks
- E.g., Image classifiers (almost working)

Generalization and Intelligence

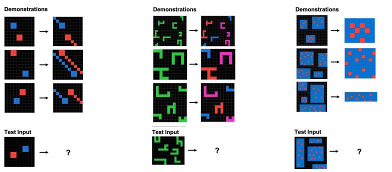
- Broad generalization
  - A.k.a., flexibility
- Adaptation to unknown unknowns across a broad category of related tasks
- E.g., self-driving cars (still sci-fi)

9/23/24

Generalization and Intelligence

- Extreme generalization
  - AGI
- Adaptation to unknown unknowns across an unknown range of tasks and domain → AI che si adatta
- ROFL (Roll On Floor Laughing)
- Not even at the horizon

Abstract scenarios



9/23/24

ARC

- Abstraction and Reasoning Corpus
- Collection of few-shot abstraction and analogy problems developed by François Chollet in 2019
  - 2020 ARC challenge: $20,000
  - 2023 ARC prize: $1,000,000+

https://arcprize.org/

AI ≠ A + I

- Artificial Intelligence is a non-compositional compound (or "idiom"): Its meaning is not the predictable sum of the meanings of the components
- Other examples of NCC
  - Military Intelligence
  - Central Intelligence (agency)

9/23/24

Artificial Intelligence

THINKING HUMANLY	THINKING RATIONALLY
ACTING HUMANLY	ACTING RATIONALLY

Artificial Intelligence

THINKING HUMANLY (NN)	THINKING RATIONALLY
ACTING HUMANLY	ACTING RATIONALLY

NN = neural network

Computational Intelligence

[redacted]

Acting Humanly	Acting Rationally

49

Computational Intelligence

- The study of how to make computers do things at which, at the moment, people are better

50

Computational Intelligence

[redacted]

- The science of making machines capable of performing tasks that would require intelligence if done by humans

51

Computational Intelligence

[redacted]

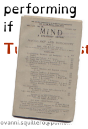
- The science of making machines capable of performing tasks that would require intelligence if done by humans
- Turing Test

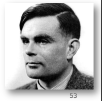
52

Intelligence

[redacted]

- The science of m king machines capable of performing tasks that would require intelligence if humans
- Tu t





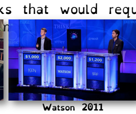
53

Computational Intelligence

[redacted]

- The science of making machines capable of performing tasks that would require intelligence

Deep Blue 1996



Watson 2011

AlphaGo 2016

54

Deep Blue was intelligent the way your programmable alarm clock is intelligent.

— Gary Kasparov, *Deep Thinking: Where Machine Intelligence Ends and Human Creativity Begins*, 2017

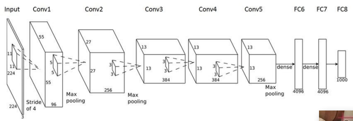
55

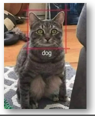
Deep Blue was intelligent the way your programmable alarm clock is intelligent.

Not that losing to a $10 million alarm clock made me feel any better.

— Gary Kasparov, *Deep Thinking: Where Machine Intelligence Ends and Human Creativity Begins*, 2017

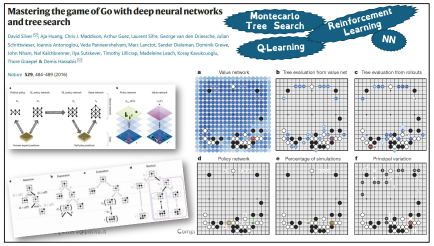
56





57

58



59

Ada Countess of Lovelace

- London, 1815–1852
- Only child of Lord Byron
- First programmer in history

1842

The "computer" has no pretensions whatever to originate anything. It can do whatever we know how to order it to perform. It can follow analysis; but it has no power of anticipating any analytical relations or truths. Its province is to assist us in making available what we are already acquainted with...

60

Computational Intelligence

- Multi-agent Systems
- Norvig's rational agents
- "Computational Intelligence is the study of the design of intelligent agents"
  (Poole et al., 1998)

Common Narratives

- Learning is Optimization
  - I.e., maximize quality over the train set in **ML**
- Optimization is Gradient Descent
  - I.e., maximize quality is minimize loss in **NN**

Computational Intelligence

- Learning is more "guess n' check"
  - *There is an obvious connection between the learning process and evolution* (Alan Turing)
- Optimization can be "trial n' error"
  - Possibly smarter than brute force's "try all"
  - Oversimplification may lead to approximate results
- Problems are solved by searching
- Solutions are built incrementally

Computational Intelligence → filosofia

- Symbolic or Sub-symbolic
- Is the origin of knowledge innate or empiric?
  - Also see *rationalism* vs. *empiricism*
- Innate knowledge?
- Innate concepts?
- Chomsky's Universal Grammar
- Neuro-Symbolic approaches

Theses

- I'm almost always looking for students interested in doing their master thesis
- Topics and constraints vary
  - Most industrial partners have tight deadlines
  - Pure research may be more relaxed
- Checkout the Telegram group!

Thesis 1

- Industrial, but with kind of "soft" deadlines
- Scenario
  - Analysis of mass spectra for diagnostic purposes (detection of **Helicobacter pylori from breath**)
  - Real data, real patients, ongoing activity
- Topics
  - Data analysis + Statistics
  - Machine learning (classification, anomaly detection, …)

Thesis 2

- Using **LLMs** in the context of **HW V&V**
- Scenario
  - Analyze and model validation/verification properties and patterns
  - Hot topic in electronic **CAD**
  - Pure research
- Topics
  - Machine learning (NN)

COMPUTATIONAL INTELLIGENCE

Politecnico di Torino
01URROV 01URRSM

2024-25

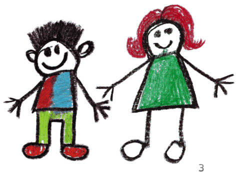
1

Computational Intelligence

https://github.com/squillero/computational-intelligence

2

Enlightening Example



3

The Six Students Problem

- Six students (three from computer engineering, three from data science and engineering) spent the afternoon studying Computational Intelligence

4

- participants : 6 { 3 computer engineers (C)
  3 data scientists (D)

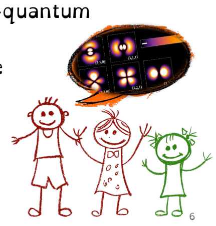
The Six Students Problem

- In the evening, they decide to go out for pizza together at Pixel Pizza, but the place is not within walking distance → problem
- Fortunately, one of the students has a tandem bicycle that can transport two people → challenge
- How will they get there?

5

Problem!

- If there are more computer engineers (C) than data scientists (D) in a group of students, the C's will start talking about post-quantum cryptography, the D's will lose interest, and the evening will be ruined → constraint



6

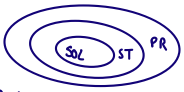
7

Modeling

- Abstraction
  - Simplifying the problem by focusing on its key elements and ignoring irrelevant details
- Representation
  - Expressing the problem in a formal language or framework

8

Modeling



- Possible terminology
  - Problem space (a.k.a., the real world) Bob, Carl, ...
  - State space (a.k.a., the result of modeling) CCC DDD
  - Solution space (a.k.a., viable ↳valid states from state space)
- Note: Some scholars adopt different terms

Modeling

- Abstraction
  - Simplifying the problem by focusing on its key elements and ignoring irrelevant details
- Representation
  - Expressing the problem in a formal language or framework
- Simplification
  - Making assumptions or approximations to make the problem more tractable

Abstraction

- A "state" in the "problem space" contains all and only the relevant information.

(friends at home, friends in pizzeria)

Representation

- Some state are invalid

(CCCDDD, ·) → tutti a casa
(CC, CDDD)
~~(DD, CCCD)~~

- A good representation exposes the constrains

count(C) <= count(D)

Representation

- Two states are connected if they differ for the position of 2 students (a trip in tandem)
- Solution: find the shortest path

(CCCDDD, ·) ⇨ (·, CCCDDD)

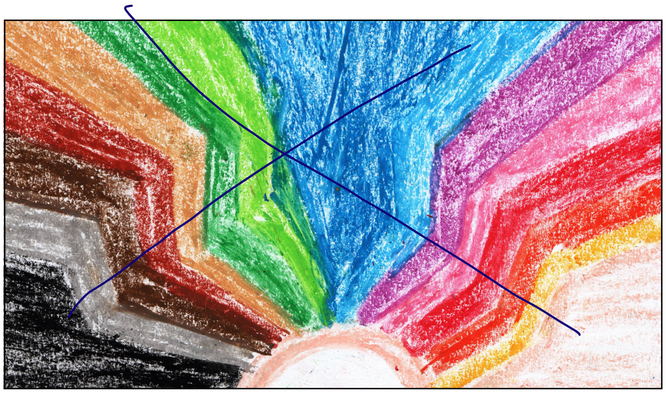
Solution

(CCCDDD, ·)
⇩
(CCDD, CD)
⇩
(CD, CCDD)
⇩
(·, CCCDDD)

Solution?

no, because there isn't the tandem

(CCCDDD, ·)
⇩
(CCDD, CD)
⇩
(CD, CCDD)
⇩
(·, CCCDDD)



Representation

- Correct representation

(friends at home, friends in pizzeria, position of the tandem)

17

Representation

- Correct representation

(friends at home, friends in pizzeria, position of the tandem)

- Correct constraints

count(C) <= count(D) or count(D) == 0

18

Representation

- Correct representation

(friends at home, friends in pizzeria, position of the tandem)

- Correct constraints

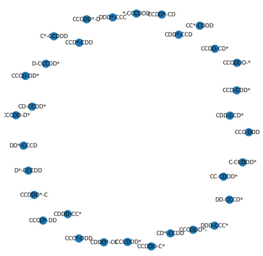
count(C) <= count(D) or count(D) == 0

- Solution: find the shortest path

(CCCDDD*, ·) ⇨ (·, CCCDDD*)

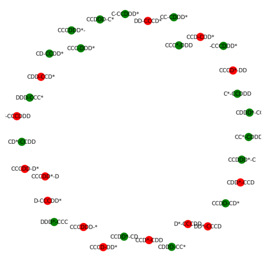
19

States



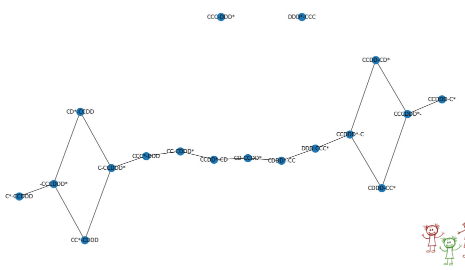
20

Valid states (Solution space)



21

Paths in the solution space



22

Solution: 11 steps

CCCDDD*- ⇨

CDDD-CC* ⇨ CCDDD*-C ⇨

DDD-CCC* ⇨ CDDD*-CC ⇨ CD-CCDD* ⇨

CCDD*-CD ⇨ CC-CDDD* ⇨ CCC*-DDD ⇨

C-CCDDD* ⇨ CD*-CCDD ⇨

-CCCDDD*

23

What did we learn?

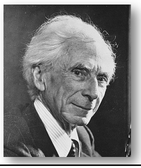
24

Modeling

- Find key elements (ignore irrelevant details)
- Formalize the problem (expose constraints)

The greatest challenge to any thinker is stating the problem in a way that will allow a solution

— Betrand Russel (1872-1970)



giovanni.squillero@polito.it Computational Intelligence

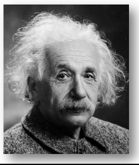
25

Modeling

- Find key elements (ignore irrelevant details)
- Formalize the problem (expose constraints)
- Make assumptions to make the problem tractable
- Approximate to reduce complexity

Everything should be as simple as possible but not simpler

— Albert Einstein (1879-1955)



giovanni.squillero@polito.it Computational Intelligence

26

Solving

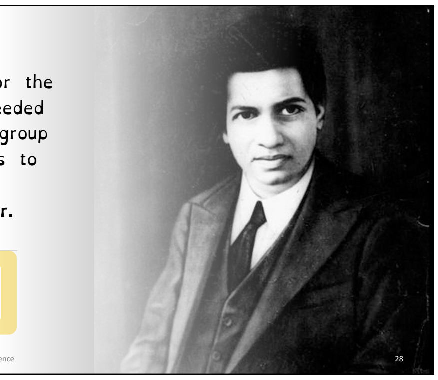
- Incrementally building a solution
  - Not necessarily starting from the beginning
- Improving an existing solution
  - This "solution" may not be feasible
- Trial and error
- Guess and check
- Generate and test (brute force)
- Learning?

giovanni.squillero@polito.it Computational Intelligence 27

27

Find the formula for the number of steps needed to bring a generic group of n_C + n_D students to the pizzeria.
Motivate the answer.

Bonus problem!
+2 at the exam



giovanni.squillero@polito.it Computational Intelligence 28

28

↳ 2024 09 25 30'

9/30/24

COMPUTATIONAL INTELLIGENCE

Politecnico di Torino

01URROV 01URRSM

2024-25

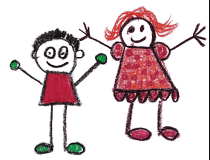
1

Heuristic classification

Non-exhaustive

Non-orthogonal

…



3

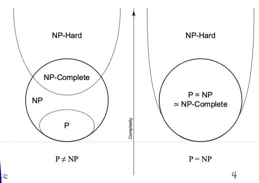
Computational Intelligence

https://github.com/squillero/computational-intelligence

2

NP Problems

- Nondeterministic Polynomial time problems
  - Verifiable in polynomial time respect to the input dim
  - Infamous P vs. NP, NP vs. co-NP dilemmas
- NP-Hard, NP-Complete
- Space/Time complexity
- Practical definition:
  - You can only test a small fraction of the input space



4

Il Dilemma:
La domanda centrale è: **P = NP?**

- **Se P = NP:** Significherebbe che ogni problema per cui possiamo verificare una soluzione rapidamente può anche essere risolto rapidamente. Questo avrebbe enormi implicazioni in campi come la crittografia, l'ottimizzazione e la teoria dei giochi.
- **Se P ≠ NP:** Indicherebbe che esistono problemi la cui soluzione può essere verificata rapidamente, ma non necessariamente trovata rapidamente.

NP vs. co-NP:

- **co-NP:** Comprende i problemi i cui **complimenti** (le risposte negative) appartengono a NP.
- **Il Dilemma:** È ancora aperto se NP è uguale a co-NP. Se NP ≠ co-NP, ci sarebbero problemi i cui complimenti non possono essere verificati rapidamente, aggiungendo un ulteriore livello di complessità.

NP-Hard:

- Questi sono problemi che sono **almeno** difficili quanto i problemi più difficili in NP. Non devono necessariamente appartenere a NP (cioè, non è richiesto che le loro soluzioni possano essere verificate in tempo polinomiale).
- **Esempio:** Il **Problema del Viaggiatore** (TSP) nella sua forma di ottimizzazione è NP-Hard.

NP-Complete:

- Sono problemi che sono **sia in NP che NP-Hard**. In altre parole, sono tra i problemi più difficili in NP e se qualcuno trovasse un algoritmo in tempo polinomiale per uno di essi, tutti i problemi in NP potrebbero essere risolti in tempo polinomiale.
- **Esempio:** Il **Problema della Soddisfacibilità Booleana (SAT)** è il primo problema dimostrato essere NP-Complete.

Importanza:
Identificare un problema come NP-Complete implica che, al momento, non esiste un algoritmo conosciuto per risolverlo rapidamente (in tempo polinomiale), e risolverlo potrebbe risolvere anche altri problemi complessi.

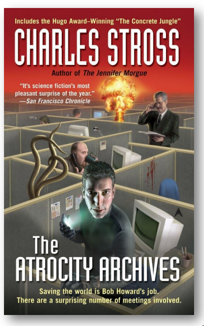
Suggested Reading

The Atrocity Archives (A Laundry Files Novel)

by Charles Stross

Mass Market Paperback (2008)

A Lovecraftian novel, mixing sci-fi with demons, a lost volume of *The Art of Computer Programming*, the Church–Turing conjecture, and all the paranormal activities made possible by P = NP…
For **real** nerds.



5

Local search algorithms

- Techniques used to find a better solution by iteratively improving an existing solution
- Useful for solving complex optimization problems where finding an exact solution is computationally expensive
- Paradigmatic example: Gradient Descent
- Caveat
  - We are only interested in the final solution (not the process by which it was obtained)

7

Black box algorithms

- The problem dynamics are unknown
  - The "black box" provides output given some input
  - Hypothesis: the Blackbox has no memory
  - The desired output is known, the task is to find the appropriate inputs
  - Constraints may not be explicit

Input → Blackbox → Output

6

Local search algorithms

- Initialization
  - Generate the current solution (random, heuristics, …)
- Main loop
  - Create neighboring solutions by tweaking (i.e., making minor adjustments) to the current solution
  - Replace the current solution with a neighboring solution
  - Keep rolling (iterations)

8

Local search algorithms

- Stochastic Gradient Descent
- Hill Climbing
- Simulated Annealing
- Tabu Search
- Genetic Algorithms
- Particle Swarm Optimization
- Nelder-Mead Simplex
- …

Example: 8 queens

- Place eight chess queens on an 8×8 chessboard so that no two queens threaten each other
- Pure constraint satisfaction
- Black box
  - Output is 1 when feasible
  - Output is the number of threats (goal 0)

What are we looking for?

- Constraint satisfaction
  - Any viable solution
- Optimization
  - The best solution
- Caveats:
  - Pure CS: minimize the number of constraint violations
  - Usually a mix. A.k.a., "constraint optimization"
  - In most practical cases we are looking for a "reasonable" solution (a.k.a., "approximate optimization")

→ greedy

Example: University timetable

- Enormously big search space
- Timetables must be good
- "Good" is defined by several competing criteria
- Timetables must be feasible
- Vast majority of search space is infeasible

Modeling problems

- Given sets of inputs & outputs, find a model that delivers correct output for every known input
  - Supervised ML: classification and regression
  - What-if scenarios
  - Artificial life

Fitness

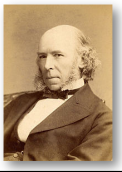
- The quantitative representation of natural and sexual selection within evolutionary biology
- Individual reproductive success
  - Average contribution to the gene pool of the next generation
  - Can be defined with respect to either a genotype or a phenotype in each environment
- Usually w or ω in population genetics models

Fitness landscapes

"Survival of the fittest"

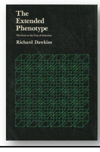
→ sopravvive chi lascia + copie di sé

- Coined by Herbert Spencer to describe Charles Darwin's original theory in his Principles of Biology (1864)
- Approved by Darwin himself as an alternative to "natural selection"



Problems

- Fitness as reproductive success
  - In Darwinian terms: "Survival of the form that will leave the most copies of itself in successive generations"
  - Survival of the fittest or Survival of the survived?
- Check out Richard Dawkins's discussion in *The Extended Phenotype* (1982)



Computer Science

- Fitness ≈ Ability to solve the given problem
  - The bigger the better
  - Optimization: "maximize the fitness"
- Often artificial
  - Real value of the solution and arbitrary penalties trying to direct the search into specific directions
- No mention to environment

Maximization vs. Minimization

- EA scholars **maximize** the fitness
- OR experts **minimize** the costs
- Economists **maximize** the reward (or value)
- Mathematicians **minimize** functions

- How to define the fitness function $f(x)$ to minimize the result of the evaluation $E(x)$?

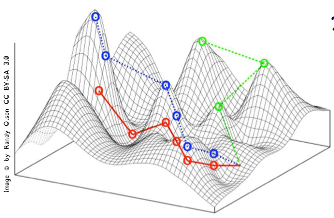
Switching Min and Max

- $f(x) = -E(x)$
  - Some algorithms may not work with negative fitness
- $f(x) = K - E(x)$
  - Some algorithms may be impaired by offsets
  - What is the optimal/safe value of K
- $f(x) = \frac{1}{E(x)}$
  - What if $E(x) = 0$?
  - Δf does not reflect ΔE

Fitness landscape

Fitness Landscapes

- Fitness landscapes can be high-dimensional, but typically are represented in 2 dimensions



3 search paths of 3 different algorithms that try to find optimum sol

Global and Local Optima

- The highest peak in the landscape is the global optimum, representing the highest fitness
- Local optima are peaks that are not as high as the global optimum, but still represent relatively high fitness (not really formal, indeed)
- Solutions are attracted by optima (basins of attraction in dynamical systems)

Global and Local Optima

- Unimodal
  - Only one peak (easy problem!)
- Multimodal
  - Several local optima
- Isolated Fitness Landscape
  - Landscapes with a relatively small number of optima

Valleys

- Valleys are regions of low fitness, they can be deep or shallow, and they can be separated by plateaus or ridges

Plateaus

- Vast areas of uniformly high fitness
- A.k.a., Mesas, as in "wandering in mesas"

Ridges

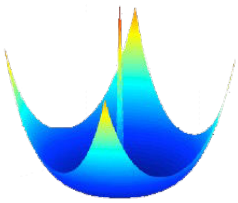
- Ridges are elevated areas that connect different peaks
- Alternative definition: sequences of local optima
- Ridges can provide pathways to explore the landscape, but can also create bottlenecks

Funnel

- Region where the fitness increases rapidly towards a particular point
- Bottlenecks?

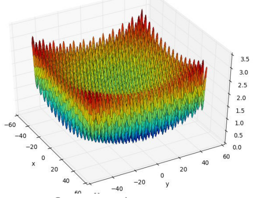
Known problems: Needle

- Needle in a Haystack (a.k.a., Telephone Pole)



Known problems: Rocky

- Noisy, Hilly, Rocky
- A landscape that have virtually no funnels



Known problems: Deception

- A fitness function that "intentionally" misleads the optimization process
- Local optima are more attractive than the global optimum
- Famous deceptive fitness landscapes:
  - Holland's New Royal Road

Fitness Landscape Analysis

- It is possible to experimentally evaluate (i.e., sample) the fitness landscape
- The topic is quite hot and not in the scope of this introductory course
  - Interested in a MS thesis? Contact Giovanni asap!

2024-25

Exploration
Exploitation

33

esplorazione vs sfruttamento

Exploration vs. Exploitation

- The **exploration vs. exploitation dilemma** is a fundamental concept in decision-making
- **Exploration**
  - Seeking out new information. Delving into the unknown to discover something new.
- **Exploitation**
  - Utilizing current knowledge to its fullest potential. Choosing the best-known option based on existing information to maximize immediate benefits.

divergenza

convergenza

34

Exploration vs. Exploitation

- Multi-Armed Bandit Problem
  - Multiple levers to pull, each with different payout percentages (average amount of money returned)
  - You don't know which lever has the highest payout
- Exploration
  - Try new levers to see which one works best
- Exploitation
  - Keep pulling the lever with the best payout
- Note: payouts can only be estimated

35

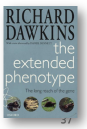
Discussion

36

Levels in biology

- Fitness: individual's ability to propagate its genes (well, almost)
- Genotype: the genetic constitution of an organism
- Phenotype: the composite of the organism's observable characteristics or traits

Richard Dawkins
The Extended Phenotype: The Long Reach of the Gene
Oxford University Press, 1982 (revised ed. 1999)



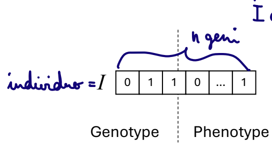
37

Levels in CS (a modest proposal)

- Fitness: how well the candidate solution is able to solve the target problem
- Genotype: the internal representation of the individual, i.e., what is directly manipulated by genetic operators
- Phenotype: the candidate solution that is encoded in the genotype
  - the intermediate form in which the genotype needs to be transformed into for evaluating fitness
  - if genotype can be directly evaluated: genotype and phenotype coincide

38

Levels (1-max) → es. di ottimizzazione nei processi evolutivi, si prende I con fitness più alta

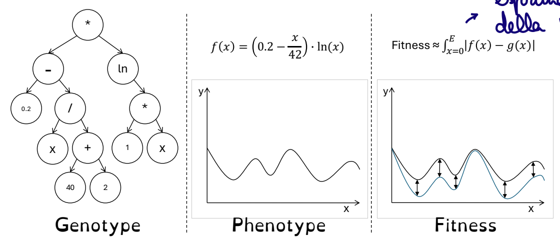


$\sum_{k=0}^{n} I(k)$ (somma di 1)

39

Levels (symbolic regression)

esprime la qualità della soluzione



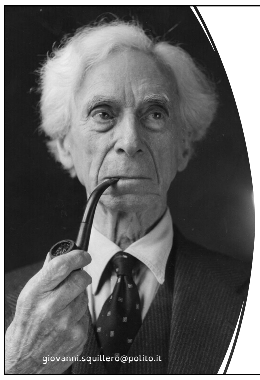
40

Discussion

- Fitness: Objective evaluation + some vague ideas about how to find a solutions (hints)
- How to evaluate a partial/incomplete solution?
- How to evaluate a non-working solution?
- Fitter is better, but is ΔF meaningful?

Discussion

- What is a "neighboring solution" in local search?
- "Genetic proximity" depends on the algorithms used (i.e., operators available):
  Is "**STRESSED**" close to "**DESSERTS**"?
  - Yes, if we have an "invert" operator
  - No, if we change character by character
- What about "**DYSTOPIA**" and "**EAGER**"?
  - Pretty close, if the gene is an index over dictionary entries



Betrand Russel

The greatest challenge to any thinker is stating the problem in a way that will allow a solution

Alternative terminology

- Modeling as "switching search spaces"
- Modeling is going "from the problem space to the fitness landscape"
- Genotypes are "mapped" to phenotypes
- The phenotype "is" the fitness (!?)

COMPUTATIONAL INTELLIGENCE

Politecnico di Torino

01URROV 01URRSM

2024-25

Computational Intelligence

https://github.com/squillero/computational-intelligence

Searching for a Solution

Hill Climbing

Hill Climbing

- Local Search
  - Start with a state (possibly illegal, but evaluable)
  - Iteratively improve/patch it exploring neighboring states
- Closely related to stochastic gradient ascent $\vec{\nabla}\uparrow$
  - But discrete and does not require a gradient
  - Iteratively test new candidate solutions in the region of a candidate solution (neighbourhood)

Hill Climbing

- Generate new candidate solutions in the region of a candidate solution (neighbourhood)
- Adopt the new ones only if they are an improvement → accept new sol only if they represent an improvement
- Quickly climb up to the (rapid convergence) first local optimum

```
S ← some initial candidate solution
repeat
    R ← Tweak(Copy(S))
    if Quality(R) > Quality(S) then
        S ← R
until S is the ideal solution or we have run out of time
return S
```


Hill Climbing

variants

1) • First-improvement Hill Climber
a.k.a., Random-Mutation Hill Climber (RMHC)

2) • Steepest-step Hill Climber
(or Steepest Ascent HC)

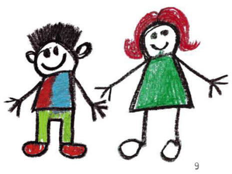
```
n ← number of tweaks desired to sample the gradient

S ← some initial candidate solution
repeat
    R ← Tweak(Copy(S))
    for n − 1 times do
        W ← Tweak(Copy(S))
        if Quality(W) > Quality(R) then
            R ← W
    if Quality(R) > Quality(S) then
        S ← R
until S is the ideal solution or we have run out of time
return S
```


+ Termination / Stopping Condition

- Best solution found
- Total number of evaluations
  - Best baseline to evaluate an algorithm
- Total number of steps
  - Useful in parallel environments
- Wall-clock time
  - Required in practical, industrial application
- Steady state (a.k.a., give up if chance of improvement is low)
  - Quite useful, often used in combination with other conditions
  - Different definitions for "steady"

TEST PROBLEMS

1-Max



One Max

- Simplest possible test problem
- The basic form are its variants are still among the most used benchmarks in the EC / soft-computing world

One Max

- Goal
  - Build a string of ones (or an array of all True, etc.)
- Fitness
  - Sum of the ones
- Very easy task:
  - Unimodal
  - Separable
  - Non deceptive
  - …

Variant: Two Max

- The goal is to maximize max(#0, #1)
  - Where #1 is number of 1's (traditional 1-max), and #0 is the number of 0's
- Useful to see when an algorithm chooses (arbitrarily) to maximize the number of 1's or of 0's

Even more variants

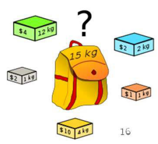
- Noisy 1-max
  - White noise is added to the evaluation of the fitness
- Dynamic 1-max
  - The optimal solution changes over time
- Weighted 1-max
  - Positions in the binary string have different weights
- Multi-objective 1-max
  - Optimizing multiple objectives simultaneously

TEST PROBLEMS

Knapsack(s)

"Classical" Knapsack prolem

- One of the best-known integer programming problems
- Given a set of items, each with a weight and a value, determine determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible



Variants

- Multidimensional Knapsack Problem
  - Handle more than one constraint, e.g., both volume limit and weight limit
  - A.k.a., Multiple-constrained Knapsack Problem
- Multiobjective Knapsack Problem
  - Target different, possibly conflicting goals, e.g., cost and sentimental value

Knapsack problems

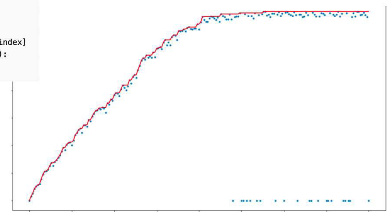
- 0-1 Knapsack problem
  - Determine determine which items to include in the collection
- Multiple Knapsack problem
  - Optimize more than one collections

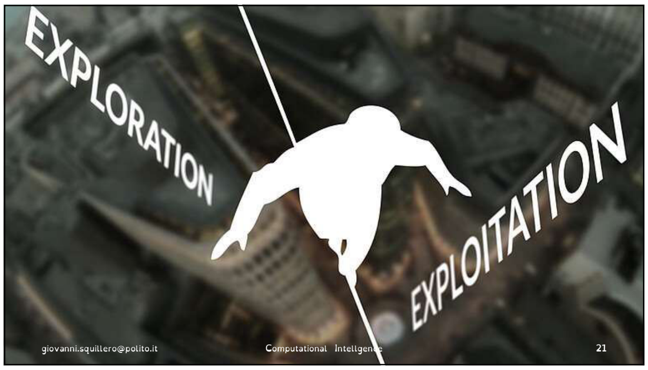
Knapsack problems

- Multidimensional Knapsack problem
  - Handle more than one constraint, e.g., both volume limit and weight limit
  - A.k.a., Multiple-constrained Knapsack Problem
- Multiobjective Knapsack problem
  - Target different, possibly conflicting goals, e.g., cost and sentimental value

Hill Climbing 0-1/multidim Knapsack

```
solution = np.full(NUM_ITEMS, False)

while True:
    new_solution = solution.copy()
    index = np.random.randint(0, NUM_ITEMS)
    new_solution[index] = not new_solution[index]
    if value(new_solution) > value(solution):
        solution = new_solution
```





Tweaks = variazione della soluzione

- Use larger sample (directly favor exploitation)
- Occasionally makes larger, more random changes (directly favor exploration)
- Accept worsening changes hoping to escape local optima (see *simulated annealing*)
- Restart from scratch from a state (always a good idea, if you have spare time)

Simulated Annealing

```
Inizio con Alta Temperatura
  ↓
Genera Soluzione Vicina
  ↓
Valuta Soluzione
  ├── Migliore → Accetta
  └── Peggiore → Accetta con Probabilità p
  ↓
Aggiorna Temperatura
  ↓
Verifica Condizioni di Terminazione
```

Simulated Annealing

- Formalized around 1980s (Kirkpatrick, et al.)
  - … but embrionic ideas date back to 1950s (Metropolis, Rosenbluth, Rosenbluth, Teller, Teller)
- Basic idea: Hill Climbing with p≠0 of accepting a worsening solution s' where $f(s) > f(s')$:

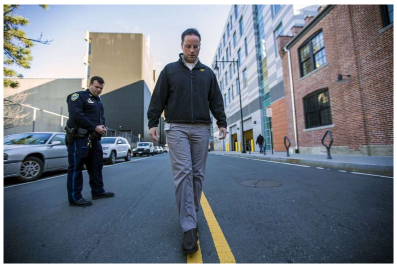
$$p = e^{-\frac{f(s)-f(s')}{t}}$$

to avoid to stop at local optimum

- Where: s is the current solution, s' is the tweaked one, and t is the temperature

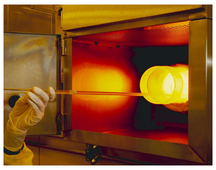
p = probability to accept this sol

Simulated Annealing (alt)



Simulated Annealing

- Schedule: the rate at which t is decreased
  - Longer schedule → more random walk (more exploration)



Tabu search


```
Inizio
  ↓
Genera Soluzioni Vicine
  ↓
Filtra Soluzioni Tabu
  ↓
Scegli Miglior Soluzione Non Tabu
  ↓
Aggiorna Lista Tabu  ──→ recent moves prohibited
  ↓
Verifica Condizioni di Terminazione
```

Tabu Search

- Tabu, from Tongan **tapu**: inviolable, forbidden
  - Commonly spelled taboo in both AmE and BrE
- Usually a variation on SAHC w/Replacement:
  - Generate n new solutions, but discard the tabu ones

```
tmp = [tweak(solution) for _ in range(NUM_SAMPLES)]
candidates = [sol for sol in tmp if is_valid(sol) and sol not in tabu_list]
if not candidates:
    continue
```

- Only works in discrete spaces!

Iterated Local Search

- A cleverer version of Hill-Climbing with Random Restarts
  - The process restart from the point selected by **new_starting_position(global, last)**
  - Almost "black magic", but simply returning the best solution found so far usually works

```
def new_starting_position(global_, last):
    if global_ is None:
        return np.random.random((NUM_SETS,)) < .5
    else:
        return global_
```

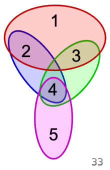

Iterated Local Search

Set cover

Set Cover

- Given a family of sets, find the subfamily of minimum cost that covers all elements in the universe
- Several possible greedy algorithms

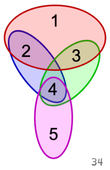
Choose the set with minimum cost ←



33

Set Cover

- Classical testbench in combinatorics, computer science, operations research, and complexity theory
- One of Karp's 21 problems shown to be NP-complete in 1972



34

- **Hill Climbing:** Un algoritmo semplice ma potente per la ricerca locale, con diverse varianti e condizioni di terminazione.
- **One Max:** Un problema di test semplice che serve come base per comprendere le dinamiche degli algoritmi di ottimizzazione.
- **Knapsack Problems:** Problemi combinatori complessi con molteplici varianti, utili per testare algoritmi di ottimizzazione in scenari reali.
- **Simulated Annealing:** Un algoritmo di ottimizzazione robusto che bilancia esplorazione e sfruttamento, evitando gli ottimi locali.
- **Tabu Search:** Un metodo avanzato che utilizza una memoria di soluzioni proibite per migliorare l'esplorazione dello spazio delle soluzioni.
- **Iterated Local Search:** Un miglioramento di Hill Climbing che utilizza ripartenze per esplorare diverse regioni dello spazio delle soluzioni.
- **Set Cover:** Un problema combinatorio NP-completo con applicazioni in vari campi, utile per valutare la potenza degli algoritmi di ottimizzazione.

10/7/24

Lab 1

- Solve efficiently these instances customizing a technique discussed in class

probabilità che un insieme abbia tutti gli elem dell'universo

Instance	Universe size	Num sets	Density
1	100	10	.2 = 20%
2	1,000	100	.2
3	10,000	1,000	.2
4	100,000	10,000	.1
5	100,000	10,000	.2
6	100,000	10,000	.3

DEADLINE: Wednesday, October 9 @ 23:59 UTC

35

Summary of classical problems of computational intelligence

Continuous search spaces

36

hill climbing × set cover problem ←

Dati di Partenza:

- **Universo**: {1, 2, 3, 4, 5}
- **Set disponibili**:
  - {1, 2}
  - {2, 3}
  - {4}
  - {5}
  - {1, 5}

Tabella del Comportamento dell'Algoritmo:

	Set Selezionati	Numero di Set	Percentuale di Copertura	Miglioramento
1	{1, 2}, {4}, {5}	3	80% (4 su 5 coperti)	No, manca l'elemento 3
2	{2, 3}, {4}, {5}	3	80% (4 su 5 coperti)	No, manca l'elemento 1
3	{2, 3}, {4}, {1, 5}	3	100% (5 su 5 coperti)	Sì, copertura completa

Descrizione dei Passi:

- **Passo 1**: Partiamo con i set {1, 2}, {4}, {5}. Questa combinazione copre gli elementi {1, 2, 4, 5}, ma lascia scoperto l'elemento 3. Copriamo quindi l'80% dell'universo.
- **Passo 2**: Sostituiamo il set {1, 2} con {2, 3}, ottenendo la copertura degli elementi {2, 3, 4, 5}. L'elemento 1 rimane scoperto, quindi la percentuale di copertura rimane all'80%, e non si osserva un miglioramento rispetto alla soluzione precedente.
- **Passo 3**: Sostituiamo il set {5} con {1, 5}. Ora i set selezionati sono {2, 3},...

10/7/24

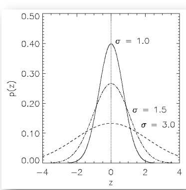
Parameter optimization

- The solution is a list of real numbers (i.e., floating point)
- Technically not Hill Climbers, but "Evolution Strategies" → for continuous space
- (1+1)-ES, (1+λ)-ES, (1,λ)-ES

37

(1+1)-ES

- A first-improvement Hill Climber were
  - tweak(·) is perfomed using a Gaussian mutation on each element
    That is: xi' = xi + N(0, σ)
- Normal distribution N(ξ,σ)
  - ξ = 0
  - σ is called "mutation step"



38

10/7/24

(1+λ)-ES

- A steepest-step Hill Climber were
  - λ is the sample size
  - tweak(·) is perfomed using a Gaussian mutation as in (1,1)-ES
- In ES slang: the plus strategy

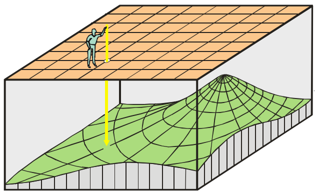
39

(1,λ)-ES

- A steepest-step Hill Climber were
  - λ is the sample size
  - tweak(·) is perfomed using a Gaussian mutation as in (1,1)-ES
  - The current solutions is always replaced, even if all the new λ solutions are worsening
- In ES slang: the comma strategy
- Caveat: using large λ reduces the risks to stop in local optimum

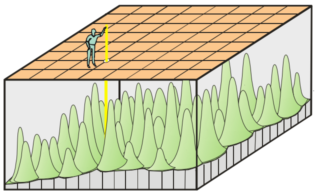
40

Strong Causality



41

Weak Causality



42

Evolution Startegies

- Normal distribution $N(\xi,\sigma)$, but how to select the best mutation step σ? → goals
  - Random guess
  - Experience
  - Meta-optimization

43

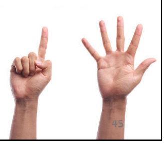
Dynamic Strategy

- Monitor the number of successes over a given amount of steps
  (called an *era* in evolutionary jargon)
- Tweak σ to balance exploration and exploitation
  - Endogenous parameter
  - Problem-space vs. algorithm space

44

1/5 success rule

- Let r_s be the ratio of successful mutations
- Reset σ after every k iterations
- Traditional values
  - $\sigma = \sigma/c$ if $r_s > 1/5$ → reduce
  - $\sigma = \sigma \cdot c$ if $r_s < 1/5$ → increase
  - $\sigma = \sigma$ if $r_s = 1/5$ → unchanged
  - With $0.8 \le c \le 1$



45

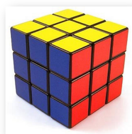
Self-adaptation

- Adaptation vs. Self-adaptation
- Self-adaptive strategy
  - Add σ to the list of parameters $\langle v_1,\dots,v_n, \sigma \rangle$
- Mutate σ and v in different steps
  - Learning rate: $\tau \propto \frac{1}{\sqrt{n}}$
  - $\sigma' = \sigma \cdot e^{\tau} \cdot N(0,1)$
  - $v'_i = N(v_i, \sigma')$
- Always mutate first σ then v_i !?

46

Self-adaptation → how to add dynamic behaviour

- Self-adaptive strategy II
  - Add a σ for each v $\langle v_1,\dots,v_n, \sigma_1,\dots, \sigma_n \rangle$
- Mutate σ and *v* as before...
- Two learning rates
  - Global learning rate τ'
  - Coordinate-wise learning rate τ'



47

Self-adaptation

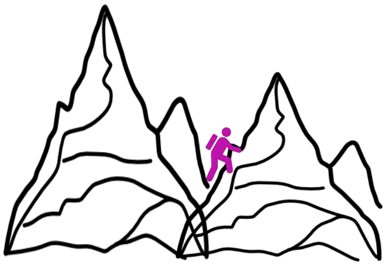
- Self-adaptive strategy III
  - $\langle v_1,\dots,v_n, \sigma_1,\dots, \sigma_n, \alpha_1,\dots, \alpha_k \rangle$ $(k = \frac{n(n-1)}{2})$
- Covariance Matrix **C**:
  - $c_{ii} = \sigma_i^2$
  - $c_{ij} = 0$ if *i* and *j* are uncorrelated
  - $c_{ij} = 1/2 \cdot (\sigma_i^2 - \sigma_j^2) \cdot \tan(2\alpha_{ij})$ when correlated

48

(μ,λ)-ES and (μ+λ)-ES

- Like (1*λ)-ES, but considering μ solutions in each iteration
  - Generate each new solution by mutating a random one among the μ existing ones
  - Plus and comma strategy as before
- Our first Evolutionary Algorithm!

Hill climber



10/7/24

Population based HC

And variable step size